

Django 教學 06

ORM與Admin

第 10～11 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [05_模型關聯與Migration](#)。

- **第 10 章：ORM 查詢與資料操作**
- **第 11 章：Admin 資料管理**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [來源索引](#) · [實作手冊對照](#)

第 10 章

ORM 查詢與資料操作

保留 shell 查詢紀錄，說明回傳型別，觀察關聯載入的查詢成本。

本章的操作環境與成果

catalog 為完整帶做範例；board／marketplace 節錄需切到對應專案。Profile／Tag 等為教學延伸。

完成成果： 保留 shell 查詢紀錄，說明回傳型別，觀察關聯載入的查詢成本。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。進階頁可回查，但所有基本驗收需完成。

10-1 Manager、QuerySet、instance

表達式	得到什麼	接著可以做什麼
Product.objects	Manager	建立查詢
Product.objects.filter(...)	QuerySet	再篩選、排序、迭代
Product.objects.get(...)	Product instance	讀欄位、save、delete
Product.objects.values(...)	字典結果的 QuerySet	讀字典鍵

get 找不到或多筆會拋例外；filter 找不到仍是空 QuerySet。
別對 instance 呼叫 filter，也別直接讀整個 QuerySet 的 name。

10-2 建立與修改：記憶體不等於已存檔

練習資料 | 在 LearnMart shell 執行

```
from marketplace.models import Category
category = Category(name="ORM 練習分類", slug="orm-lab")
category.full_clean()
category.save()
category.name = "ORM 練習分類 (已更新) "
category.save(update_fields=["name"])
category.refresh_from_db()
```

第二次重跑前先處理既有 orm-lab 分類，否則 unique 驗證會失敗。
objects.create() 可直接建立並儲存，但不會自動替你 full_clean()。

10-3 建立商品前先取得必要關聯

LearnMart shell | 先確認 seed_demo 已建立 seller 帳號與 tech 分類。

```
from marketplace.models import Category, Product, User
```

```
seller = User.objects.get(username="seller")
```

```
category = Category.objects.get(slug="tech")
```

```
product = Product.objects.create(  
    seller=seller, category=category,  
    name="教學鍵盤", description="練習 ORM",  
    price=990, stock=5,  
)
```

`.create()` 會立即 INSERT 並回傳已儲存 instance；成功後 `product.pk` 有值。

注意：一般 `.create()` / `.save()` 不等同自動呼叫完整 `full_clean()`。

10-4 刪除單筆練習商品

接續上一頁，只操作剛建立且尚未被訂單引用的商品。

```
practice_pk = product.pk
product.delete()
Product.objects.filter(pk=practice_pk).exists() # False
```

- `.delete()` 是執行刪除的動作；`on_delete` 決定關聯對象被刪除時的規則。
- 刪商品不會反過來刪除它的分類或賣家。
- 若商品已被 `OrderItem` 保護，刪除會受阻；回查 3-4 的關聯圖。

驗收：查不到練習商品，原分類與賣家仍存在。批次刪除差異見 5-10。

10-5 篩選、排除、排序與切片

```
from marketplace.models import Product
qs = (Product.objects
      .filter(is_active=True, stock__gt=0)
      .exclude(category__slug="archived")
      .order_by("price", "pk"))
first_ten = qs[:10]
```

同一 filter 的多個條件預設 AND；exclude 排除符合條件者。

先排序再取前 10 筆，才有明確的「前」。

一般切片形成 LIMIT／OFFSET；不要在切片之後繼續 filter 或重排。

10-6 常用 lookup 與 Q

寫法	意義
<code>name__icontains="鍵盤"</code>	包含字串，大小寫細節依 backend
<code>price__gte=100、price__lte=1000</code>	含端點的範圍條件
<code>category__slug__in=["tech", "books"]</code>	屬於候選值集合
<code>shipped_at__isnull=True</code>	沒有出貨時間

```
from django.db.models import Q
Product.objects.filter(Q(name__icontains=q) | Q(description__icontains=q))
```

q 是搜尋字串；Q 用 `|` 表達 OR，不是 Python 的 or。

10-7 values 與 values_list 的結果

```
Product.objects.values("id", "name")  
# 每列像 {"id": 17, "name": "鍵盤"}  
Product.objects.values_list("id", "name")  
# 每列像 (17, "鍵盤")  
Product.objects.values_list("name", flat=True)  
# 每列像 "鍵盤"
```

這些仍是 QuerySet，但每筆結果不是 Product instance。

flat=True 只適用單一欄位；字典與 tuple 不能呼叫 model.save()。

需要匯出欄位時很好用；需要物件行為時保留 model instances。

10-8 關聯查詢與 distinct

教學延伸 | 完成 Tag 模型後

```
Product.objects.filter(tags__name__in=["二手", "可議價"]).distinct()
```

同一商品可能符合兩個標籤，中介表 JOIN 因此產生重複商品列。
distinct 用來去重；先理解 JOIN 為何增加列數，再決定是否需要它。

現有案例：`Message.objects.filter(author__username="amy")`。
雙底線沿關聯走到欄位；它不是 Python 屬性存取的小數點。

10-9 Lazy evaluation 與查詢成本

```
qs = Product.objects.filter(is_active=True) # 組查詢
print(qs.query)                             # 觀察 SQL
rows = list(qs)                             # 執行並取資料
```

- iteration、list、len、bool 等會觸發評估；QuerySet 的 repr 也可能查詢。
- 只問有沒有，用 exists；只要數量，用 count。
- 馬上還要遍歷所有結果時，先 exists 再遍歷可能多查一次。
- QuerySet 有結果快取，但新組出的查詢不能假設共享舊快取。

10-10 N+1：列表逐筆讀取關聯的成本

```
products = Product.objects.all()
for product in products:
    print(product.seller.username)
```

可能發生：

- 1 次查 products
- 每個 product 再查 seller
- 20 商品可能約 21 次 query

問題不是 Python loop 本身，而是每次讀尚未載入的 relation 都觸發 database access。

先觀察問題，再選擇 optimization。

10-11 關聯預載取決於資料形狀

抽入 LearnMart 原第 5 章的列表／詳情案例

```
Product.objects.filter(is_active=True).select_related("category", "seller")  
Product.objects.prefetch_related("reviews__author")
```

JOIN 是把關聯資料表連接查詢；select_related 用 JOIN 載入單值關聯，例如 FK／一對一。prefetch_related 分批查詢再組合，適合反向 FK／多對多。

Admin 列表也會讀關聯；list_select_related 可處理相同問題。先觀察實際 query 數，不要把所有 relation 都一律預載。

10-12 aggregate 與 annotate

```
from django.db.models import Avg, Count
Product.objects.aggregate(avg_price=Avg("price"))
# 整份集合 → 一個摘要字典
Product.objects.annotate(review_count=Count("reviews"))
# 每個 Product → 多一個 review_count attribute
```

集合摘要與每列計算不同；計算 attribute 不是新增資料庫欄位。
Product.average_rating 是現有 property，逐件呼叫可能逐件查詢。
多個關聯一起聚合時要注意 JOIN 造成重複計數，需驗證結果。

10-13 商品平均評分的查詢行為

目前 LearnMart 實作的注意點 | `Product.average_rating`

一般 `@property` 不會自動 cache。若 template 在 `{% if product.average_rating %}` 與輸出時各讀一次，背後的 `aggregate()` 也可能執行兩次。

補充／進階 | 只評估一次的 template 寫法

```
{% with rating=product.average_rating %}
  {% if rating %}
    <span class="text-warning">★ {{ rating }}</span>
  {% endif %}
{% endwith %}
```

`prefetch_related("reviews__author")` 不會讓 `aggregate()` 自動改用 prefetched rows；需要依實際 query 測量再設計。

10-14 進階 | 更新、刪除與原子運算

教學延伸 | 以一筆練習商品操作

```
from django.db.models import F
changed = Product.objects.filter(pk=product.pk, stock__gte=1).update(
    stock=F("stock") - 1,
)
```

條件與扣減在同一個 SQL UPDATE；changed 為 0 表示沒符合條件的資料。
這不等於完整結帳交易；多表一致性仍需第 19 章的 transaction 設計。
update 不逐筆呼叫 save；批次 delete 也不呼叫每個 model 的 delete 方法。
刪除前確認篩選範圍與關聯保護；只在練習資料上操作。

10-15 get_or_create 與 update_or_create

現有 LearnMart | CartItem 與 Review 的重複規則

- get_or_create 回傳 (instance, created) ； defaults 用於新建。
- update_or_create 找到就更新，沒找到才建立。
- 查詢鍵需與資料庫唯一約束配合，不能只靠方法名稱避免競態。

```
review, created = Review.objects.update_or_create(  
    product=product, author=user,  
    defaults={"rating": 5, "comment": "操作清楚"},  
)
```

product、user 要先取得；評分是否合法仍需驗證。

10-16 seed_demo 重跑會發生什麼？

LearnMart 現有初始化指令 | 節錄帳號建立流程

```
seller, created = User.objects.get_or_create(
    username="seller", defaults={"role": User.Role.SELLER},
)
if created:
    seller.set_password("seller12345")
    seller.save()
```

- 帳號已存在時，defaults 不會重設 role，密碼也不會重設。
- 分類與商品也用 get_or_create 避免反覆新增相同查詢鍵的資料。
- 重跑能補缺少的資料，不能把既有資料庫還原成固定初始狀態。

10-17 ORM 操作練習（25 分鐘）

先用 LearnMart 練習資料，在 shell 完成：

1. 查詢有庫存商品，依價格與 pk 排序，取前 5 筆。
2. 只取名稱與價格，說明回傳的是物件、字典還是 tuple。
3. 查特定分類的商品，列出賣家帳號，比較預載前後的 query 數。
4. 依本章建立與刪除商品的範例 建立、修改並刪除練習商品，確認資料庫結果。

驗收：交查詢程式、實際結果與回傳型別；空資料時要能說明原因。

多對多加分題：查兩個標籤並解釋 distinct 的用途。

10-18 ORM 先在 shell 小步驗證

```
uv run python manage.py shell
```

```
from board.models import Message  
Message.objects.count()  
Message.objects.all()[:3]
```

先確認 QuerySet 做得到，再把它放進 view。

10-19 Shell：查詢、修改與刪除的獨立練習

回到 django_lab，啟動 shell；建立一件可刪除的臨時商品：

```
from decimal import Decimal
from catalog.models import Category, Product
category, _ = Category.objects.get_or_create(
    slug="stationery", defaults={"name": "文具"})
product = Product.objects.create(category=category,
    name="刪除練習商品", price=Decimal("50.00"), stock=3)
print(Product.objects.filter(stock__gt=0))
print(category.products.all())
product.stock = 5
product.save()
product.refresh_from_db()
print(product.stock) # 5
product.delete()
```

只刪除剛建立的臨時商品；分類與其他商品仍存在。

第 10 章實作與離堂檢核

任務： 保留 shell 查詢紀錄，說明回傳型別，觀察關聯載入的查詢成本。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnBoard 01 原第 5 章](#)；[LearnMart 01 原第 5 章](#)。手冊保留原章號，對照表列出本課位置。

第 11 章

Admin 資料管理

完成可搜尋、可篩選的 Admin，使用不同帳號驗證管理權限。

本章的操作環境與成果

catalog 為完整帶做範例；board／marketplace 節錄需切到對應專案。Profile／Tag 等為教學延伸。

完成成果： 完成可搜尋、可篩選的 Admin，使用不同帳號驗證管理權限。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。進階頁可回查，但所有基本驗收需完成。

11-1 Admin 在系統中的角色

Admin 根據 Model 與設定產生內部管理介面。

- 可以先管理分類、商品與留言，不必先完成每張自訂表單。
- 它讀寫同一個資料庫，不是獨立的示範資料集。
- Model 定義資料，ModelAdmin 定義管理介面。
- 購物車、結帳、賣家工作流程仍使用專用 View，接續第 18～20 章。

登入後能看到資料，不等於所有使用者都應該拿到管理權限。

11-2 第一次啟動 Admin

在 LearnMart 根目錄執行，先確認環境與依賴已同步

```
uv run python manage.py migrate
uv run python manage.py createsuperuser
uv run python manage.py runserver
```

開啟 `http://127.0.0.1:8000/admin/`，使用剛建立的帳號登入。

專案需有 `admin`/`auth`/`sessions` 等 app、相應 middleware 與 template 設定。

兩個專案已配置；路由為 `path("admin/", admin.site.urls)`。

驗收：能登入；但自訂模型還需註冊才會出現在列表。

11-3 最小註冊與 ModelAdmin

```
# app 的 admin.py ; 以下兩種擇一  
admin.site.register(Category)
```

```
from django.contrib import admin  
from .models import Product  
  
@admin.register(Product)  
class ProductAdmin(admin.ModelAdmin):  
    list_display = ("name", "price", "stock")
```

同一 model 不要重複註冊。現有 ProductAdmin 已存在，練習時修改原 class。
新增 model 要 migration；只改列表欄位通常不需要。

11-4 LearnBoard：留言列表與搜尋

抽入現有 MessageAdmin，節錄

```
@admin.register(Message)
class MessageAdmin(admin.ModelAdmin):
    list_display = ("id", "author", "short_content", "created_at", "updated_at")
    search_fields = ("content", "author__username")
    empty_value_display = "訪客"

    @admin.display(description="內容")
    def short_content(self, obj):
        return obj.content[:30]
```

short_content 是顯示方法，不是資料庫欄位；它不會截短原始留言。
author__username 讓搜尋沿外鍵找作者；empty_value_display 是空值顯示設定。

11-5 LearnMart：商品列表的三個設定

抽入現有 ProductAdmin

```
@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = ("name", "category", "seller", "price", "stock", "is_active")
    list_filter = ("category", "is_active")
    search_fields = ("name", "description")
```

list_display 決定列表欄；list_filter 提供側邊篩選；search_fields 決定搜尋範圍。

操作：搜尋「鍵盤」→ 篩選上架 → 點入一筆商品。

列表沒顯示的欄位，仍可能出現在編輯表單。

11-6 編輯表單：fields、fieldsets、readonly

教學延伸 | 加入既有 ProductAdmin

```
readonly_fields = ("created_at", "updated_at")
fieldsets = (
    ("商品", {"fields": ("name", "category", "seller", "description")}),
    ("販售", {"fields": ("price", "stock", "is_active", "image")}),
    ("紀錄", {"fields": ("created_at", "updated_at")}),
)
```

fields 指定平面順序；fieldsets 做分組，兩者不要同時設定。
auto_now 類欄位不可編輯，可透過 readonly_fields 顯示。
readonly 是 Admin 介面規則，不是資料庫不可修改的保證。

11-7 Admin 的 slug 輸入提示

教學延伸 | 先以本段取代 Category 的簡單註冊

```
@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ("name", "slug")
    search_fields = ("name", "slug")
    prepopulated_fields = {"slug": ("name",)}
```

移除原 `admin.site.register(Category)`，避免 `AlreadyRegistered`。

新增時會用 JavaScript 協助填 slug；不是 `model.save` 的生成規則。

先用英文名稱 Daily Tools 試驗；中文 slug 需另設計 `allow_unicode` 與路由。

11-8 關聯選單、autocomplete 與查詢

教學延伸 | 加入既有 ProductAdmin

```
autocomplete_fields = ("category", "seller")  
list_select_related = ("category", "seller")  
list_per_page = 30
```

- autocomplete 的目標 Admin 必須設定 search_fields，並有相應存取權限。
- CategoryAdmin 可用上一頁；UserAdmin 原本已有帳號搜尋設定。
- list_select_related 處理列表讀 FK 的查詢成本。
- 少量資料可用普通選單；多對多亦可考慮 filter_horizontal。

先區分「編輯頁選對象」與「列表頁讀對象」的兩種查詢需求。

11-9 訂單 Inline：同頁看父子資料

抽入現有 LearnMart | 節錄

```
class OrderItemInline(admin.TabularInline):
    model = OrderItem
    extra = 0
    readonly_fields = ("product_name", "unit_price", "quantity", "seller")

@admin.register(Order)
class OrderAdmin(admin.ModelAdmin):
    inlines = [OrderItemInline]
```

OrderItem 的 order 外鍵把明細連到當前訂單；extra=0 只是不預留空白列。
現有設定不是完整唯讀：product 仍可編輯，新增／刪除也需另看權限。
不要把「部分 readonly」當成訂單歷史已被完整保護。

11-10 訂單明細唯讀的教學設計

教學延伸 | 替換上一頁的 Inline 定義

```
class OrderItemInline(admin.TabularInline):  
    model = OrderItem  
    extra = 0  
    fields = ("product", "product_name", "unit_price", "quantity", "seller")  
    readonly_fields = fields  
    can_delete = False  
  
    def has_add_permission(self, request, obj=None):  
        return False
```

這限制本 Inline；父訂單欄位及其他寫入入口仍要各自設計。
若要修改訂單，應透過明確的取消／調整流程，保留歷史與一致性。

11-11 staff、superuser、Group 與模型權限

設定	意義
is_active+is_staff	預設 Admin 的登入資格
is_superuser	預設權限系統下擁有全部權限
view／add／change／delete	每個模型的不同操作權限
Group	組合一群人的權限

staff 不等於能管理所有模型；賣家 role 也不等於 staff。

建一個只含 view_product 的群組，交給一般 staff 試登入。

驗收：能看商品，但不能新增、修改與刪除；不要用 superuser 驗收限制。

11-12 進階 | 權限與資料範圍

模型權限回答「能否改 Product」；不自動回答「只能改自己的 Product」。

若要讓 staff 只管理指定資料，至少考慮：

- `get_queryset` 限制列表可見範圍。
- `has_view_permission`／`has_change_permission` 等限制物件操作。
- 關聯欄位的可選資料，不能只隱藏列表。
- 自訂 action 與直接網址也要受相同規則約束。

本課讓 Admin 服務內部管理員；賣家自己的介面在第 17 章用 View 實作。

11-13 現有 UserAdmin 為何另有一個 class ?

現有 LearnMart | MarketplaceUserAdmin，節錄

```
from django.contrib.auth.admin import UserAdmin

@admin.register(User)
class MarketplaceUserAdmin(UserAdmin):
    fieldsets = UserAdmin.fieldsets + (
        ("市集身份", {"fields": ("role",)}),
    )
```

繼承 UserAdmin 保留使用者管理與密碼處理；再加入 role。
新增使用者的畫面另用 add_fieldsets；現有專案也有設定。
不要用普通字串欄位方式直接改 password；使用既有密碼修改介面。

11-14 進階 | Admin action：批次下架

教學延伸 | 放進既有 ProductAdmin class 內

```
actions = ["deactivate"]

@admin.action(description="將選取商品下架", permissions=["change"])
def deactivate(self, request, queryset):
    count = queryset.update(is_active=False)
    self.message_user(request, f"已下架 {count} 件商品")
```

queryset 是使用者選取的資料；此處只改上架旗標。

update 不觸發逐筆 save／auto_now；若需更新時間，必須明確指定。

結帳、出貨與退款不適合只改一個欄位，應呼叫完整業務流程。

11-15 Admin 操作實驗（35 分鐘）

在 LearnMart 練習分支依序操作：

1. 登入 Admin，建立「ORM 練習分類」，輸入 slug。
2. 修改 ProductAdmin，加入時間唯讀與關聯預載。
3. 為現有商品選分類、填價格、上傳圖片；回前台看結果。
4. 用名稱搜尋、分類篩選；比較列表與編輯頁欄位。
5. 建立只有 view_product 的 staff 帳號，驗證操作限制。

驗收：後台畫面＋前台資料一致；超出位數的價格／重複 slug 都能顯示錯誤。
關聯對象的「+」按鈕是否出現，取決於目標模型註冊與使用者權限。

11-16 整合 migration 與 Admin 的精選欄位

接續第 9 章的 is_featured 練習，在同一個 LearnMart 分支操作。

1. 確認已閱讀並套用新增欄位的 migration。
2. 在現有 ProductAdmin 的 list_display、list_filter 加入 is_featured。
3. 將一件商品設為精選，檢查列表顯示與篩選結果。
4. 執行 makemigrations --check，確認沒有遺漏的模型變更。

ProductForm.Meta.fields 目前不含 is_featured，前台商品表單不會自動新增此欄。

配套手冊： LearnMart [原第 4 章](#)、[原第 5 章](#)； LearnBoard [原第 4 章](#)、[原第 5 章](#)。

11-17 Admin 常見問題排查

現象	優先檢查
登入被拒絕	密碼、is_active、is_staff
登入後看不到模型	app 安裝、模型註冊、模型權限
AlreadyRegistered	同一 model 重複註冊
autocomplete 系統檢查錯誤	目標 Admin 註冊與 search_fields
時間欄位放 fields 後報錯	是否需加入 readonly_fields
圖片有資料卻顯示 404	storage 檔案、MEDIA_URL、開發路由

先執行 `uv run python manage.py check`，再根據錯誤定位設定。

11-18 綜合設計：商品、標籤與管理後台

分組交付三份可檢查的成果：

- **設計表**：各欄位型別、pk／unique、NULL 與預設值理由。
- **操作紀錄**：一對一／一對多／多對多的正反向查詢與結果型別。
- **管理介面**：列表、搜尋、篩選、唯讀時間、staff 權限的驗收畫面。

每組互問：如果改名字、刪帳號、重複送出、沒上傳圖片，會發生什麼？
能用資料規則解釋結果，比只展示「畫面可以開」更重要。

11-19 離堂檢核

1. pk 與 unique 的目的有什麼不同？
2. blank=True 能否保證 DB 接受 NULL？
3. auto_now 與 default=timezone.now 有什麼差別？
4. FK 應放在哪一方？一對一的反向結果是什麼？
5. 多對多 clear 會刪掉對方物件嗎？
6. Model.Meta、ModelForm.Meta、ModelAdmin 各管理哪一層？
7. save 是否自動 full_clean？staff 是否擁有全部模型權限？

請各選一題，用 LearnBoard 或 LearnMart 的具體例子回答。

11-20 官方參考：Model 與 ORM

以 Django 6.1 文件為教材基準：

- Models：模型與關聯
- Field reference：欄位與參數
- Meta options：排序、表名、索引
- Constraints：唯一性與檢查約束
- Making queries：查詢與寫入
- Model instances：full_clean 與 save

11-21 官方參考：關聯、檔案與 Admin

- Many-to-many：關聯操作範例
- Time zones：時間與時區
- File uploads：上傳資料
- Admin site：註冊、列表、表單與 Inline
- Admin actions：批次操作
- Authentication：使用者與權限

查文件先確認版本，再定位到類別、參數與使用限制。

第 11 章實作與離堂檢核

任務： 完成可搜尋、可篩選的 Admin，使用不同帳號驗證管理權限。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnBoard 01 原第 4 章](#)；[LearnMart 01 原第 4 章](#)。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[07_資料列表搜尋與分頁](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。
- 原始教材與合併去向見 [來源索引](#)。